Estructura de datos

Algoritmos de ordenamiento

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM
Universidad Autónoma de la Ciudad de México
NADA HUMANO ME ES AJENO

Índice

1 Bubble Sort

2 Insertion Sort

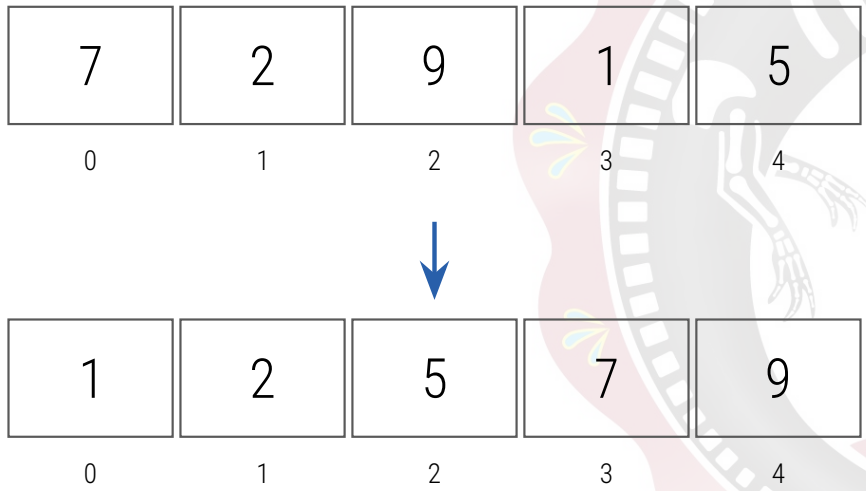
3 Quick Sort

4 Merge Sort

¿Qué es un algoritmo de ordenamiento?

Definición

Un **algoritmo de ordenamiento** reorganiza los elementos de una **colección** de acuerdo con un criterio de comparación.



- Orden **ascendente** o **descendente.**
- Comparaciones e intercambios.
- **Tiempo de ejecución** y memoria utilizada.
- **Estabilidad** del ordenamiento.

Clasificación de los algoritmos

Algoritmos iterativos

Definición

Utilizan **ciclos** para recorrer y reorganizar los elementos.

Ejemplo:

- Bubble Sort.
- Insertion Sort.

Divide y vencerás

Definición

Dividen el problema en subproblemas que se resuelven **recursivamente.**

Ejemplo:

- Quick Sort.
- Merge Sort.

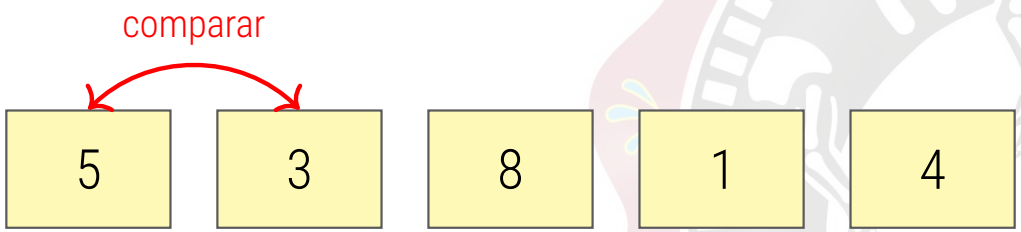
Pregunta

¿Todos los algoritmos requieren el mismo **número de comparaciones** y **movimientos**?

Bubble Sort: concepto

Ordenamiento por burbuja

Compara elementos adyacentes y los **intercambia** cuando se encuentran en el orden incorrecto.



$5 > 3 \quad \Rightarrow$ intercambiar

- Se recorre el arreglo **varias veces.**
- En cada pasada, el mayor elemento pendiente **llega al final.**
- Se repite hasta que el arreglo está ordenado.

Bubble Sort: algoritmo

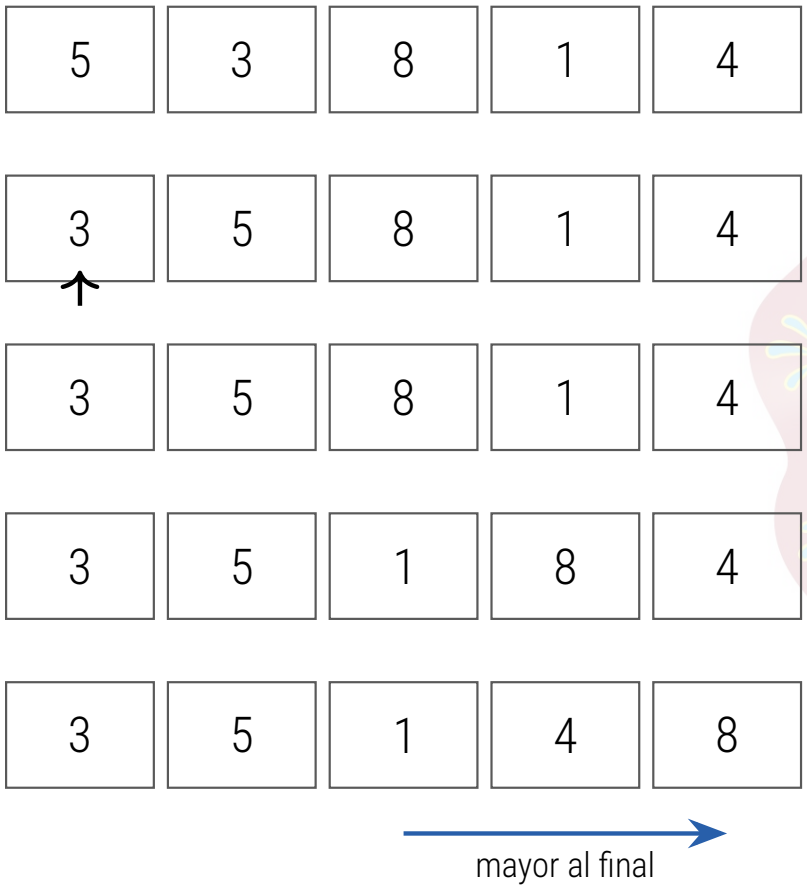
Entrada: Arreglo A de n elementos
Salida: Arreglo A ordenado ascendentemente

```
1:  for i ← 0 hasta n − 2 do
2:      intercambio ← falso
3:      for j ← 0 hasta n − 2 − i do
4:          if A[j] > A[j + 1] then
5:              aux ← A[j]
6:              A[j] ← A[j + 1]
7:              A[j + 1] ← aux
8:              intercambio ← verdadero
9:          end if
10:     end for
11:     if intercambio = falso then
12:         terminar ciclo
13:     end if
14: end for
```


Idea principal

En cada pasada se coloca el elemento mayor de la sección no ordenada en su posición definitiva.

Bubble Sort: primera pasada



Estado inicial

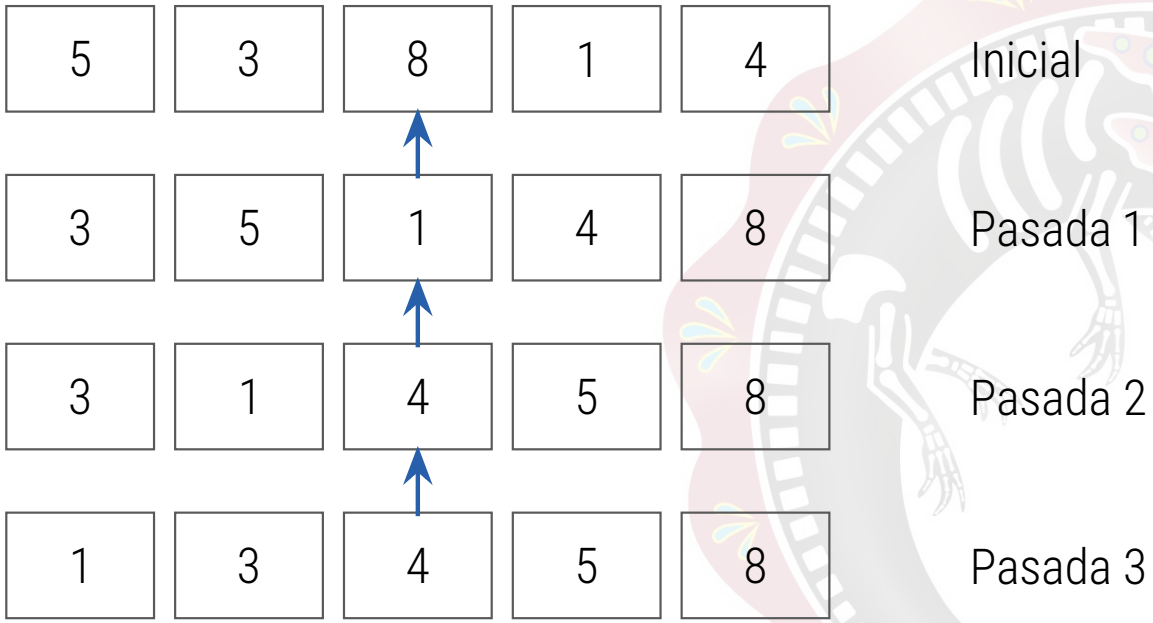
$5 > 3$: intercambiar

$5 < 8$: conservar

$8 > 1$: intercambiar

$8 > 4$: intercambiar

Bubble Sort: pasadas sucesivas



¡Importante!

Después de cada pasada, al menos un elemento queda en su posición definitiva.

Bubble Sort: implementación en Java I

```java
public static void bubbleSort(int[] datos) {
  int n = datos.length;
  for (int i = 0; i < n - 1; i++) {
    boolean intercambio = false;
    for (int j = 0; j < n - 1 - i; j++) {
      if (datos[j] > datos[j + 1]) {
        int aux = datos[j];
        datos[j] = datos[j + 1];
        datos[j + 1] = aux;
        intercambio = true;
      }
    }

    if (!intercambio) {
      break;
    }
  }
}
```

Bubble Sort: implementación en Java II

Bubble Sort: análisis

Caso	Complejidad
Mejor caso	$O(n)$
Caso promedio	$O(n^2)$
Peor caso	$O(n^2)$
Memoria adicional	$O(1)$

Ventajas

- Implementación sencilla.
- No requiere arreglos auxiliares.
- Es estable en la implementación mostrada.

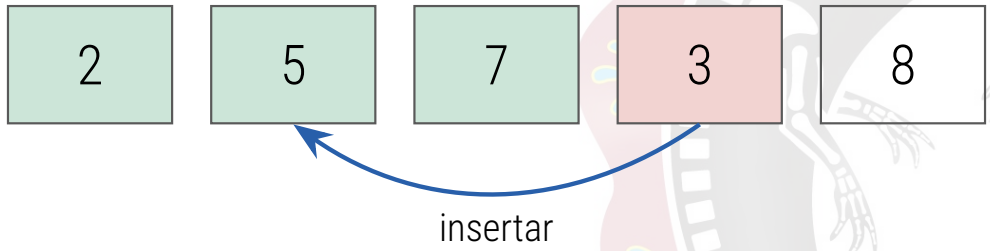
¡Importante!

Resulta poco eficiente para arreglos grandes. El mejor caso $O(n)$ requiere la detección de pasadas sin intercambios.

Insertion Sort: concepto

Ordenamiento por inserción

Construye progresivamente una sección ordenada del arreglo, insertando cada nuevo elemento en la posición correspondiente.



- El primer elemento se considera ordenado.
- Se selecciona el siguiente elemento.
- Se desplazan los mayores hacia la derecha.
- Se inserta el elemento en el espacio disponible.

Insertion Sort: algoritmo

Entrada: Arreglo A de n elementos
Salida: Arreglo A ordenado ascendentemente

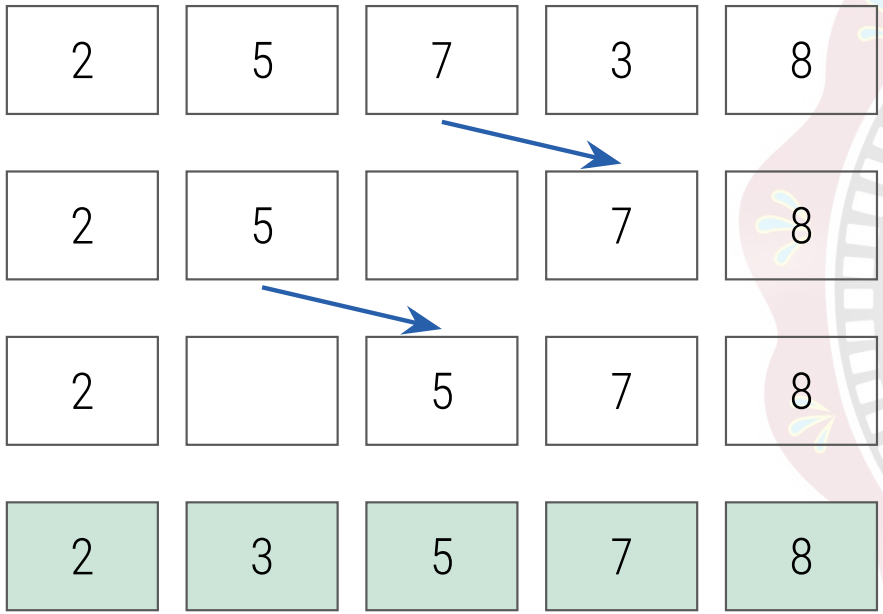
1: **for** $i \leftarrow 1$ **hasta** $n-1$ **do**
2: $\quad clave \leftarrow A[i]$
3: $\quad j \leftarrow i-1$
4: $\quad$ **while** $j \geq 0$ **y** $A[j] > clave$ **do**
5: $\quad\quad A[j+1] \leftarrow A[j]$
6: $\quad\quad j \leftarrow j-1$
7: $\quad$ **end while**
8: $\quad A[j+1] \leftarrow clave$
9: **end for**



Idea principal

Se mantiene una sección ordenada del arreglo y cada nuevo elemento se inserta en la posición que le corresponde.

Insertion Sort: desplazamiento e inserción



Insertion Sort: implementación en Java

```java
public static void insertionSort(int[] datos) {
  for (int i = 1; i < datos.length; i++) {
    int clave = datos[i];
    int j = i - 1;

    while (j >= 0 && datos[j] > clave) {
      datos[j + 1] = datos[j];
      j--;
    }

    datos[j + 1] = clave;
  }
}
```

Invariante del ciclo

Antes de cada iteración del ciclo externo, el subarreglo desde 0 hasta $i - 1$ se encuentra ordenado.

Insertion Sort: análisis

Caso	Complejidad
Mejor caso	$O(n)$
Caso promedio	$O(n^2)$
Peor caso	$O(n^2)$
Memoria adicional	$O(1)$

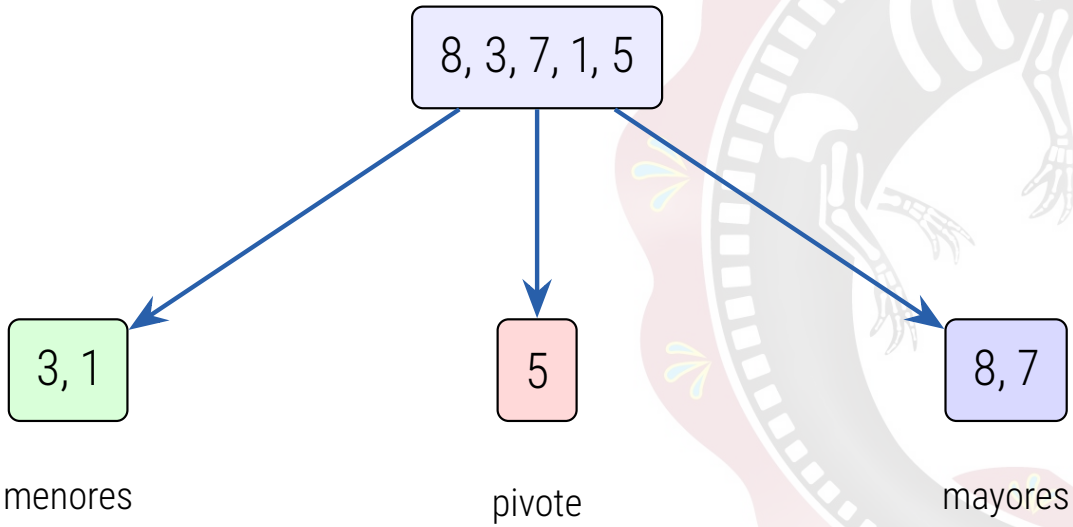
Ventajas

- Es sencillo de implementar.
- Funciona bien con arreglos pequeños.
- Es eficiente cuando los datos están casi ordenados.
- Es estable y ordena en el mismo arreglo.

Quick Sort: concepto

Divide y vencerás

Quick Sort selecciona un pivote y reorganiza el arreglo en dos particiones: elementos menores o iguales al pivote y elementos mayores que él.



Idea

Una vez ubicado el pivote en su posición definitiva, se ordenan recursivamente las dos particiones.

Quick Sort: algoritmo principal

Entrada: Arreglo A, índices $inicio$ y fin
Salida: Subarreglo $A[inicio \dots fin]$ ordenado

```
1: procedure QuickSort(A, inicio, fin)
2:     if inicio < fin then
3:         p ← Particionar(A, inicio, fin)
4:         QuickSort(A, inicio, p − 1)
5:         QuickSort(A, p + 1, fin)
6:     end if
7: end procedure
```



Divide y vencerás

1. Seleccionar un pivote y particionar.
2. Ordenar recursivamente la sección izquierda.
3. Ordenar recursivamente la sección derecha.

Quick Sort: algoritmo de partición

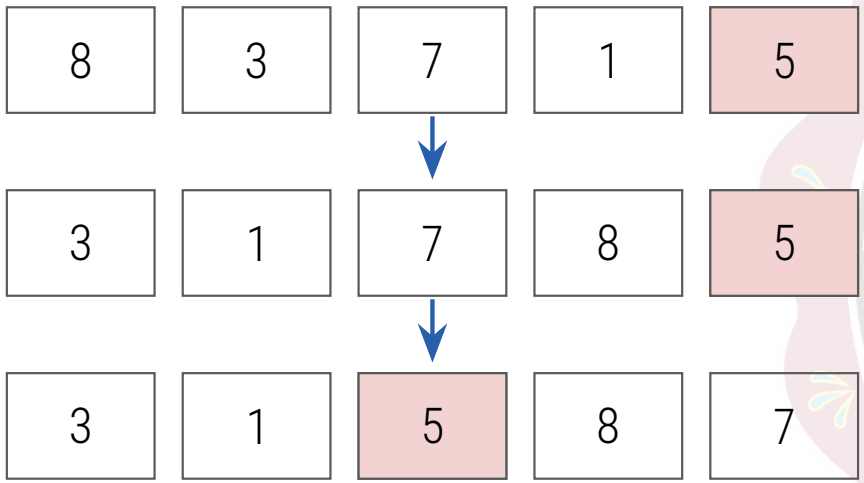
```
1:  procedure PARTICIONAR(A, inicio, fin)
2:      pivote ← A[fin]
3:      i ← inicio − 1
4:      for j ← inicio hasta fin − 1 do
5:          if A[j] ≤ pivote then
6:              i ← i + 1
7:              Intercambiar A[i] y A[j]
8:          end if
9:      end for
10:     Intercambiar A[i + 1] y A[fin]
11:     return i + 1
12: end procedure
```


Propiedad

Después de particionar, el pivote se encuentra en su posición definitiva dentro del arreglo.

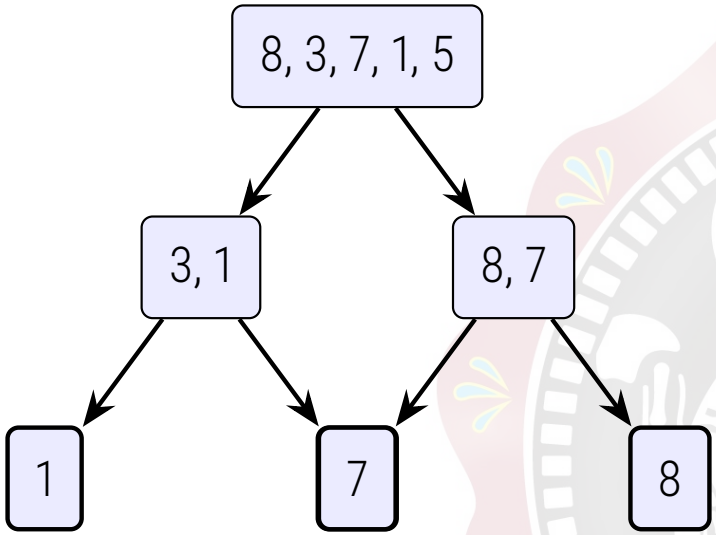
Quick Sort: partición con pivote final

Utilizaremos como pivote el último elemento y el esquema de partición de Lomuto.



$$[3, 1] \quad | \quad \boxed{5} \quad | \quad [8, 7]$$

Quick Sort: árbol de recursión



Caso base

Un subarreglo con cero o un elemento ya está ordenado.

Importante

El costo de Quick Sort depende de qué tan equilibradas resulten las particiones.

Quick Sort: implementación en Java I

```
public static void quickSort(
int[] a, int inicio, int fin) {
  if (inicio >= fin) {
    return;
  }

  int p = particionar(a, inicio, fin);

  quickSort(a, inicio, p - 1);
  quickSort(a, p + 1, fin);
}

private static int particionar(
int[] a, int inicio, int fin) {
  int pivote = a[fin];
  int i = inicio - 1;

  for (int j = inicio; j < fin; j++) {
```

Quick Sort: implementación en Java II

```
      if (a[j] <= pivote) {
        i++;
        intercambiar(a, i, j);
      }
    }

    intercambiar(a, i + 1, fin);
    return i + 1;
}

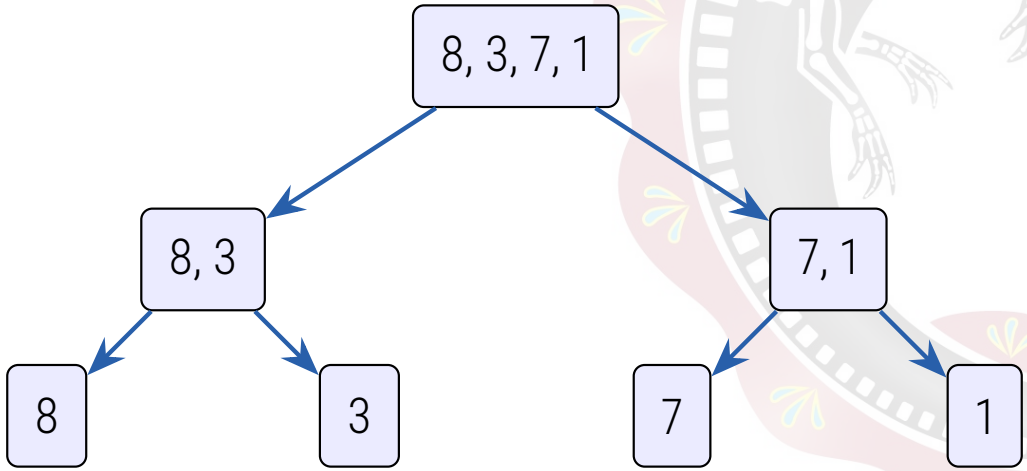
private static void intercambiar(
int[] a, int i, int j) {
    int aux = a[i];
    a[i] = a[j];
    a[j] = aux;
}
```



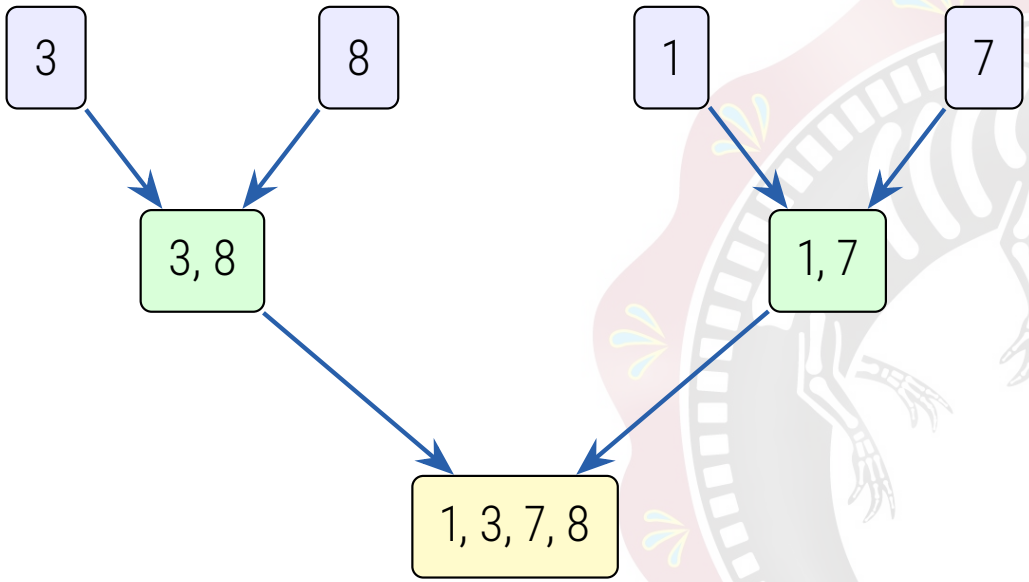
Merge Sort: concepto

Ordenamiento por mezcla

Merge Sort divide recursivamente el arreglo en mitades hasta obtener subarreglos de un elemento. Después los fusiona manteniendo el orden.



Merge Sort: proceso de fusión



Fusión

Se comparan los primeros elementos disponibles de ambos subarreglos y se copia el menor al arreglo resultado.

Merge Sort: algoritmo principal

Entrada: Arreglo A, índices $inicio$ y fin
Salida: Subarreglo $A[inicio \dots fin]$ ordenado

1: **procedure** MERGESORT($A, inicio, fin$)
2: **if** $inicio < fin$ **then**
3: $medio \leftarrow inicio + \left\lfloor \frac{fin - inicio}{2} \right\rfloor$
4: MERGESORT($A, inicio, medio$)
5: MERGESORT($A, medio + 1, fin$)
6: FUSIONAR($A, inicio, medio, fin$)
7: **end if**
8: **end procedure**

Divide y vencerás

El arreglo se divide recursivamente hasta obtener subarreglos de un elemento. Posteriormente, se fusionan manteniendo el orden.

Merge Sort: algoritmo de fusión I

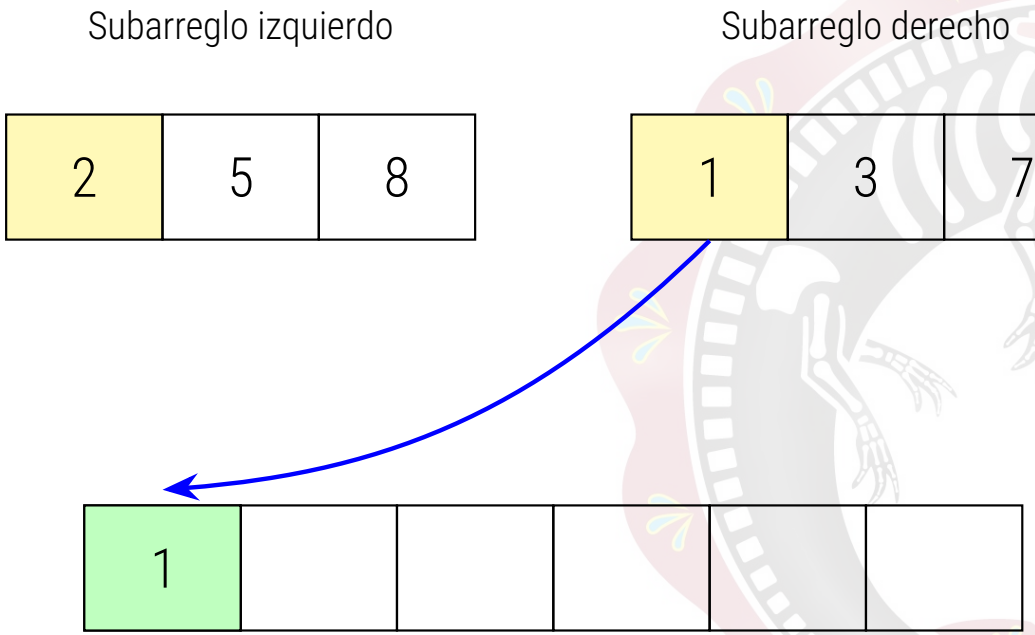
```
1:  procedure FUSIONAR(A, inicio, medio, fin)
2:      Crear arreglo auxiliar B de tamaño fin − inicio + 1
3:      i ← inicio
4:      j ← medio + 1
5:      k ← 0
6:      while i ≤ medio y j ≤ fin do
7:          if A[i] ≤ A[j] then
8:              B[k] ← A[i]
9:              i ← i + 1
10:         else
11:             B[k] ← A[j]
12:             j ← j + 1
13:         end if
14:         k ← k + 1
15:     end while
16:     while i ≤ medio do
17:         B[k] ← A[i]
```



Merge Sort: algoritmo de fusión II

```
18:            i ← i + 1
19:            k ← k + 1
20:        end while
21:        while j ≤ fin do
22:            B[k] ← A[j]
23:            j ← j + 1
24:            k ← k + 1
25:        end while
26:        for t ← 0 hasta k − 1 do
27:            A[inicio + t] ← B[t]
28:        end for
29:    end procedure
```

Merge Sort: funcionamiento de la fusión



Se copia el menor elemento disponible

$$1 < 2 \quad \Rightarrow \quad \text{copiar } 1$$

Merge Sort: implementación en Java I

```
public static void mergeSort(
int[] a, int inicio, int fin) {

  if (inicio >= fin) {
    return;
  }

  int medio = inicio + (fin - inicio) / 2;

  mergeSort(a, inicio, medio);
  mergeSort(a, medio + 1, fin);

  fusionar(a, inicio, medio, fin);
}

private static void fusionar(
int[] a, int inicio, int medio, int fin) {
```

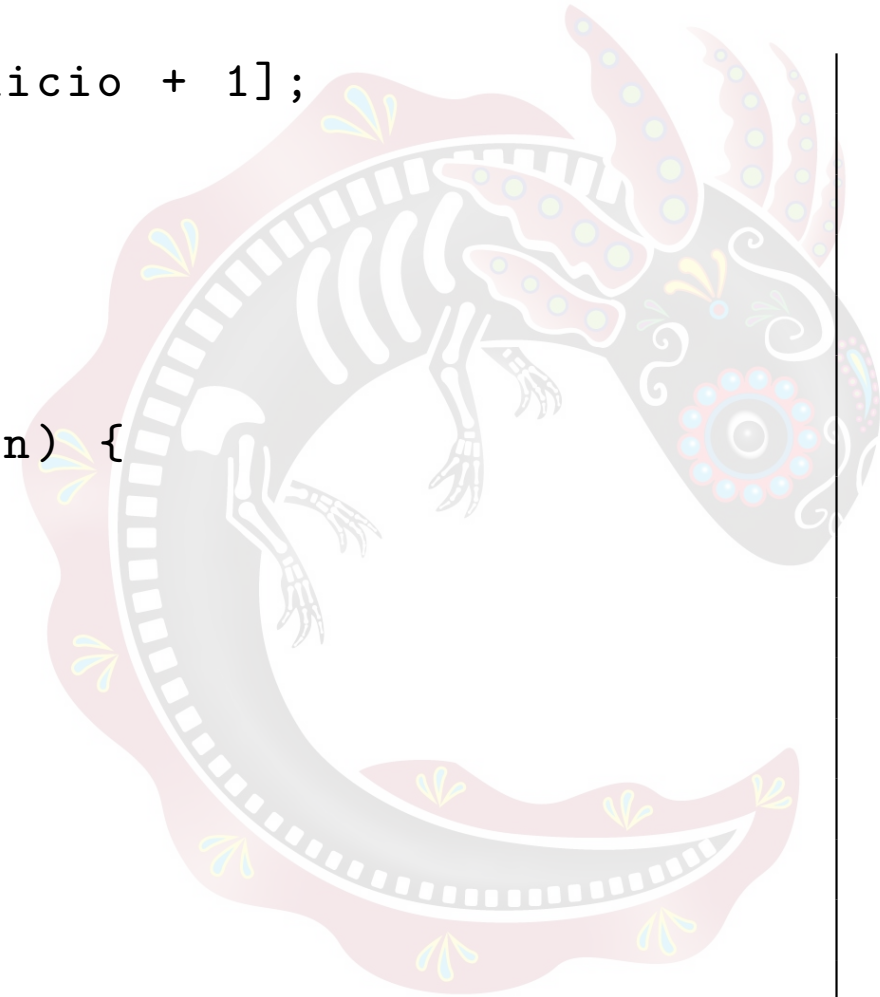
Merge Sort: implementación en Java II

```
int[] aux = new int[fin - inicio + 1];

int i = inicio;
int j = medio + 1;
int k = 0;

while (i <= medio && j <= fin) {
  if (a[i] <= a[j]) {
    aux[k++] = a[i++];
  } else {
    aux[k++] = a[j++];
  }
}

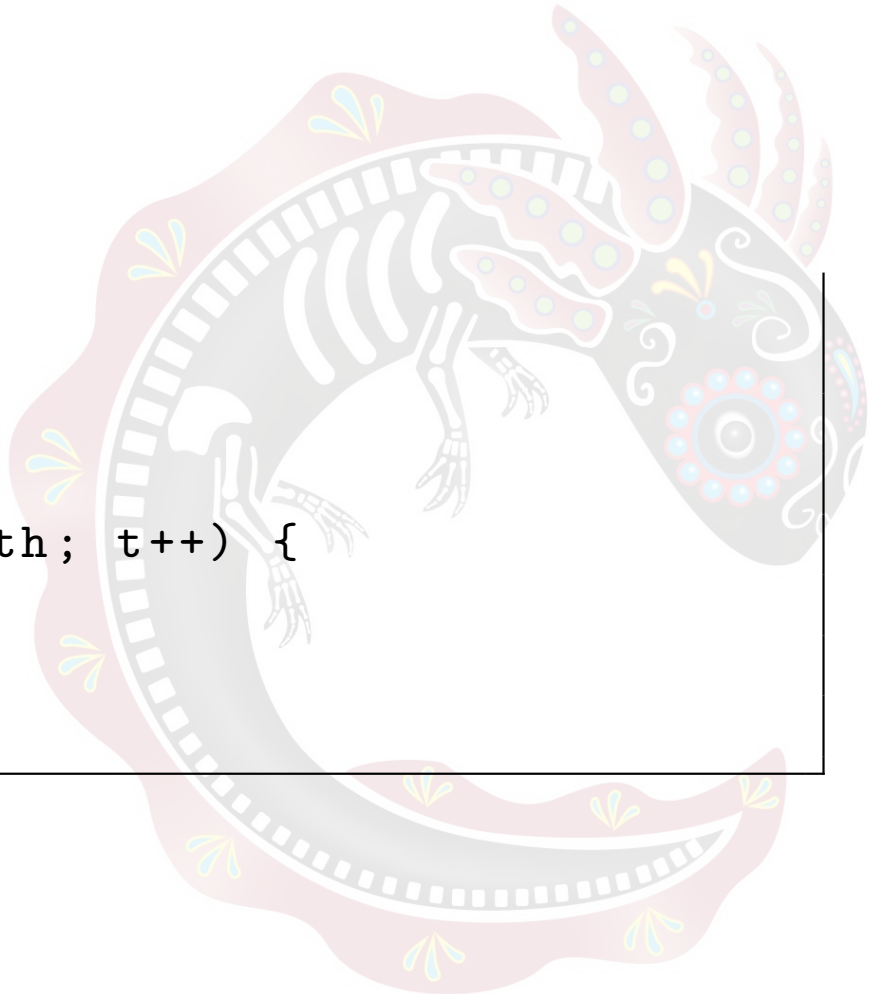
while (i <= medio) {
  aux[k++] = a[i++];
}
```



Merge Sort: implementación en Java III

```
    while (j <= fin) {
      aux[k++] = a[j++];
    }

    for (int t = 0; t < aux.length; t++) {
      a[inicio + t] = aux[t];
    }
}
```



Comparación de algoritmos

Algoritmo	Mejor	Promedio	Peor	Memoria extra
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

* Memoria promedio de la pila recursiva en la implementación con particiones in situ; en el peor caso puede llegar a $O(n)$.

Conclusiones

- **Bubble Sort:** sencillo, pero generalmente lento.
- **Insertion Sort:** adecuado para arreglos pequeños o casi ordenados.
- **Quick Sort:** eficiente en promedio, pero sensible a la elección del pivote.
- **Merge Sort:** garantiza $O(n \log n)$, aunque requiere memoria auxiliar.

Fin de la presentación



¿Preguntas o comentarios?